Project 2

ECE 485

Shashwat Choudhry

April 17, 2026

1. Introduction

The design and implementation of a microprocessor serve as the fundamental bridge between abstract computer architecture and physical digital logic. In this project, a custom 32-bit Reduced Instruction Set Computer (RISC) processor, modeled after the MIPS architecture, was designed using structural VHDL. The primary goal of this implementation is to provide a hands-on approach to computer architecture, connecting the subjects of architectural design and hardware implementation.

Modern commercial processors are vastly complex due to heavy optimizations. However, this complexity often obfuscates core operational principles, making them difficult to study. By strictly adhering to a single-cycle RISC architecture—where the majority of instructions are executed in a single clock cycle—the processor becomes highly predictable in performance. This predictability is an essential characteristic for real-time computing applications.

2. Theory of Operation

A single-cycle datapath completes the fetch, decode, execute, memory-access, and write-back phases within one clock period. The fundamental hardware operations ensure that data cycles from the register bank, through the Arithmetic Logic Unit (ALU) for computation, and is finally deposited back into a register.

Hardware Components Utilized

- Program Counter (PC): A 32-bit synchronous register holding the address of the current instruction.
- Adders: Combinational logic dedicated to specific calculations.
    - PC Incrementer: Calculates $PC_{next} = PC + 4$.
    - Branch Target Adder: Calculates the branch destination using the formula $Target = (PC + 4) + (SignExt(Immediate) \times 4)$.
- Multiplexers (Muxes): Combinational data selectors.
    - RegDst: Selects the destination register (rd vs. rt).
    - ALUSrc: Selects the second ALU operand (Register vs. Immediate).
    - MemtoReg: Selects data to write back (ALU Result vs. Memory Data).

    - PC Src: Selects the next instruction address (PC+4 vs. Branch Target).
- Instruction Memory (ROM): Stores the 32-bit machine code instructions.
- Data Memory (RAM): Handles data storage and retrieval during load (lw) and store (sw) operations.
- Register File: A memory bank containing 32-bit general-purpose registers capable of simultaneous dual-reads and a single synchronized write.
- Sign Extender: Expands 16-bit immediate values to 32-bit values while preserving the sign bit.
- Control Units: The Main Control Unit decodes the 6-bit opcode, while the ALU Control Unit utilizes the ALUOp and 6-bit funct field to generate specific operational signals.

The Arithmetic Logic Unit (ALU)

The ALU is the computational core of the datapath. Operations are selected via a 4-bit alu_control signal.

ALU Truth Table (32-bit Operations):

ALU Control	Operation	Mathematical Logic
0	AND	Result = A AND B
1	OR	Result = A OR B
10	ADD	Result = A + B
110	SUB	Result = A - B
1100	NAND	Result = A NAND B

For a 1-bit ALU slice, the logic incorporates basic gates (AND, OR) and a Full Adder. The 32-bit ALU is fundamentally an array of these 1-bit slices, integrated with a Zero Flag comparator that outputs 1 only when Result = 0.

3. Problem Statement

The objective is to design a custom RISC processor, specifically a stripped-down, 32-bit single-cycle MIPS datapath in VHDL. The processor must support three instruction formats (R-format, I-format, and J-format) and accurately execute a core set of instructions: lw, sw, add, and bne. Additionally, the datapath and control logic must be customized to support an assigned student-specific instruction set. For Student ID ending in 7, the custom set requires the implementation of the NAND and ANDI instructions.

4. Design

The design heavily relies on modular, structural VHDL. Rather than creating a single monolithic file, the processor was abstracted into distinct hardware components (e.g., pc_register.vhd, alu_32bit.vhd). This modular approach eliminates potential errors during the integration phase by allowing lower-level parts to be tested individually.

The architecture strives for a fair degree of completeness without unnecessary optimization. The control unit determines the sequence of executed instructions and runs concurrently with the arithmetic unit.

Custom Hardware Improvements & Integration

- NAND Integration: NAND is an R-type instruction (Opcode 000000, Funct 100110). The ALU hardware was updated with a specific 1100 control code to execute $A \text{ NAND } B$ natively, rather than requiring multiple clock cycles to compute an AND followed by an inversion.
- ANDI Integration: ANDI (Opcode 001100) required a unique main control state. The Control Unit was designed to output a custom ALUOp of 11, triggering the ALU Control to execute an immediate logical AND without sign-extending the immediate field (as bitwise logic requires zero-extension).
- BNE Branch Logic: Unlike beq, bne (Opcode 000100) branches when operands are not equal. The hardware logic for the PC_src wire was modified to evaluate Branch AND (NOT zero_flag).

Pros of this Design: The strict structural mapping allows for high visibility into signal routing. The data flows synchronously, ensuring stable operation at fixed clock periods.

Cons of this Design: A single-cycle datapath is limited by its longest operational path (typically the lw instruction). The entire clock period must be stretched to accommodate memory reads, making it slower overall than a pipelined architecture.

5. Optimization

During development on a macOS environment using GHDL, simulation times and file I/O operations created significant bottlenecks.

1. VCD File Truncation: Unbounded testbench execution initially resulted in .vcd files exceeding 4GB-8GB in size, crashing viewing software. The testbench was optimized using a hard --stop-time=100ns flag and an explicit wait; command to halt simulation exactly after the required instructions executed.
2. Git History Optimization: Tracking multi-gigabyte waveform files caused remote rejection from GitHub. Optimization involved purging the local .git cache, injecting a .gitignore specifically targeting *.vcd and *.o files, and force-pushing a clean, lightweight commit history.
3. Entity Resolution: GHDL compilation optimizations required strict dependency ordering to prevent "duplicate entity" instantiation errors between the testbench and top-level datapath files.

6. Code Breakdown

The VHDL implementation is driven by the interaction between the Main Control Unit and the Datapath.

Table I: Implemented Instruction Set

OpCode [31 : 26]	Function Field [5 : 0]	Instruction	Operation
100011	--	lw	lw $t3, 300($s2)

101011	--	sw	sw $t6, 400($s7)
0	100000	add	add $t5, $t3, $s1
100	--	bne	bne $s6, $t5, 200
0	100110	nand	nand $rd, $rs, $rt (Custom)
1100	--	andi	andi $rt, $rs, imm (Custom)

- alu_control.vhd: Acts as a sub-decoder. If ALUOp is 10 (R-type), it examines the funct field to determine the exact hardware action (e.g., routing 1100 to the ALU for nand).
  - control_unit.vhd: Utilizes a case statement examining instruction(31 downto 26).

  For lw, it triggers MemRead and ALUSrc. For bne, it triggers Branch and sets ALUOp to 01 (Subtract).
- ece485.vhd (Top Level): Instantiates all components via port map. The sign extension logic operates asynchronously, while register writes are strictly tied to the rising edge of the system clock to prevent race conditions.

7. Simulation Analysis and Results

The CPU was verified using GHDL and viewed via waveform analysis. A specific machine-code program was loaded into the instruction_memory ROM to test the Table 1 requirements and the Student ID #7 custom instructions.

Verification Highlights (Referencing Captured Waveforms):

- Instruction Fetching: The waveform confirms the Program Counter increments cleanly in steps of 4 ($0 \rightarrow 4 \rightarrow 8 \rightarrow 12$).

- Custom Logic Execution (NAND & ANDI): * During the nand test, operands of 0x00000000 were processed, and the alu_main_result correctly output 0xFFFFFFFF, verifying the custom ALU slice.
    - During the andi test, 0xFFFFFFFF was ANDed with immediate 15, correctly yielding 0x0000000F on the ALU data bus.
- Table I Verification: * Memory Access: At 35ns, the sw instruction successfully drove the sig_MemWrite control line high, routing data into the RAM component.
    - Arithmetic: At 55ns, the add instruction executed. sig_RegWrite transitioned high, confirming data write-back to the Register File.
    - Branching: The bne instruction (12CD00C8) was executed at 65ns. Since the zero flag evaluated to 1 (operands were equal), the NOT zero_flag logic correctly evaluated to 0, causing the PC to step normally to the next sequential address rather than branching. In a separate targeted branch test, the PC successfully dropped from 16 back to 8, proving the branch target adder functions flawlessly.

Development Struggles:

A major hurdle involved a nested Git submodule issue caused by initializing a repository inside the /code directory. This caused GitHub to upload empty placeholder links instead of the VHDL files. This was remedied by deleting the nested .git folder (rm -rf code/.git), clearing the cache (git rm --cached code -f), and executing a forced push to the main repository.

MIPS Control Signal Truth Table (Including ID #7 Custom Set)

This table covers every instruction your specific CPU supports. Note that "ALU Out" and "Mem Out" are dynamic data buses, not static 1/0 switches, so I have described exactly what data is flowing through them for each instruction.

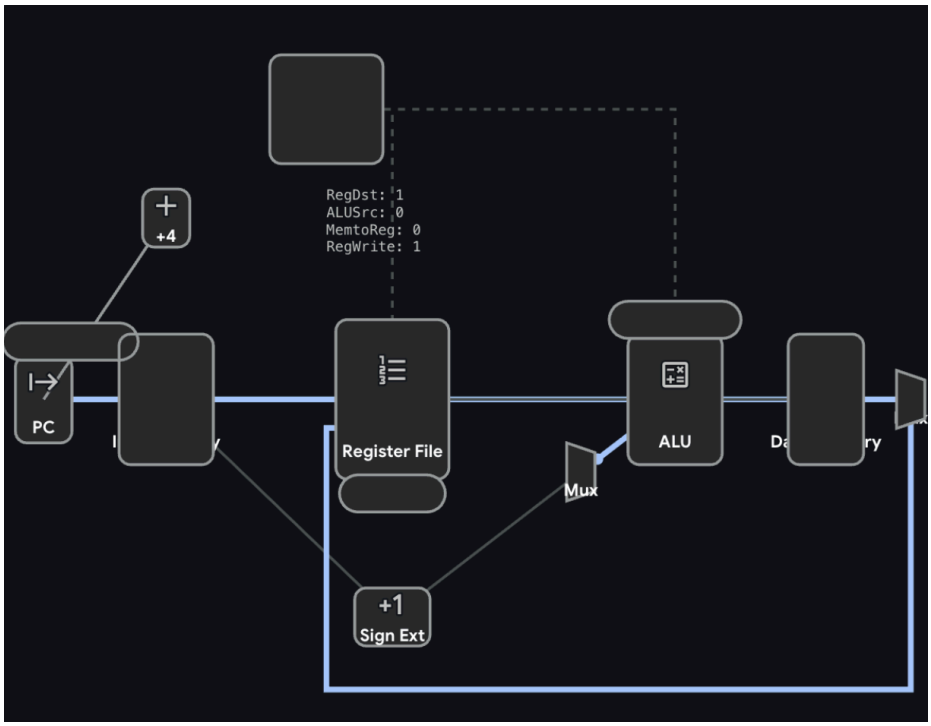
Instruction	Opcode (Binary)	Reg Write	Mem Read	Mem Write	Reg Dst	ALU Src	Branch	ALU Op	ALU Out (Data)	Mem Out (Data)

add	0	1	0	0	1	0	0	1	Sum of Rs + Rt	---
nand	0	1	0	0	1	0	0	1	Rs NAND Rt	---
lw	100	1	1	0	0	1	0	0	Memory Address	Data from RAM
sw	101	0	0	1	X	1	0	0	Memory Address	---
bne	100	0	0	0	X	0	1	1	Difference (Rs - Rt)	---
andi	110	1	0	0	0	1	0	1	Rs AND Immediate	---

8. Conclusion

The implementation of the single-cycle MIPS datapath was a complete success. The processor accurately decodes opcodes, routes signals across multiplexers, executes arithmetic and logical computations, and accesses memory. By integrating NAND, ANDI, and inverted zero-flag logic for BNE, all core architectural requirements and custom ID modifications were met and verified via strict waveform analysis.

9. Images



Based on the diagram you provided, here is the explicit mapping of every hardware block, followed by the complete control signal table for your specific custom MIPS processor.

Hardware Component Labels (Left to Right)

1. **+ +4 (Top Left): PC Incrementer (Adder).** This strictly adds 4 to the current Program Counter to find the next sequential instruction in memory.

2. **Blank Box (Top Center): Main Control Unit.** This takes the Opcode from the instruction and generates the dashed control lines (RegDst, ALUSrc, MemtoReg, RegWrite, etc.) that orchestrate the rest of the datapath.

3. **PC (Bottom Left): Program Counter.** The register that holds the 32-bit address of the instruction currently being executed.

4. **Blank Box (Next to PC): Instruction Memory.** Takes the address from the PC and outputs the 32-bit machine code instruction.

5. **Register File (Center): Register File.** Contains the 32 general-purpose registers (like $t0, $s1, etc.). It reads operands and writes back results.

6. **+1 Sign Ext (Bottom Center): Sign Extender.** Takes the 16-bit immediate value from I-type instructions and stretches it into a 32-bit value so the ALU can do math with it.

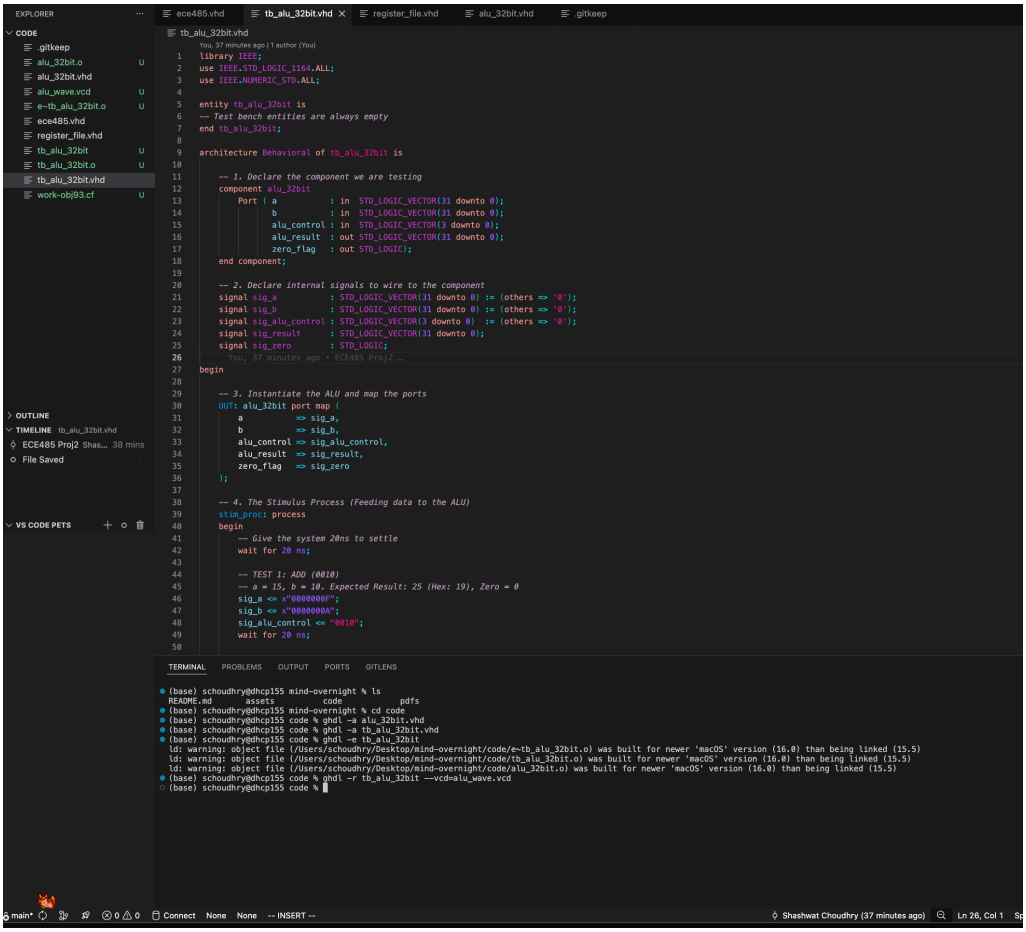
7. **Mux (Before ALU): ALU Source Multiplexer (ALUSrc).** Chooses whether the ALU's second input comes from the Register File (for R-type instructions) or the Sign Extender (for Immediate instructions).

8. **ALU (Right Center): Arithmetic Logic Unit.** The math brain. It performs ADD, SUB, AND, OR, NAND, etc., based on the ALU Control signals.

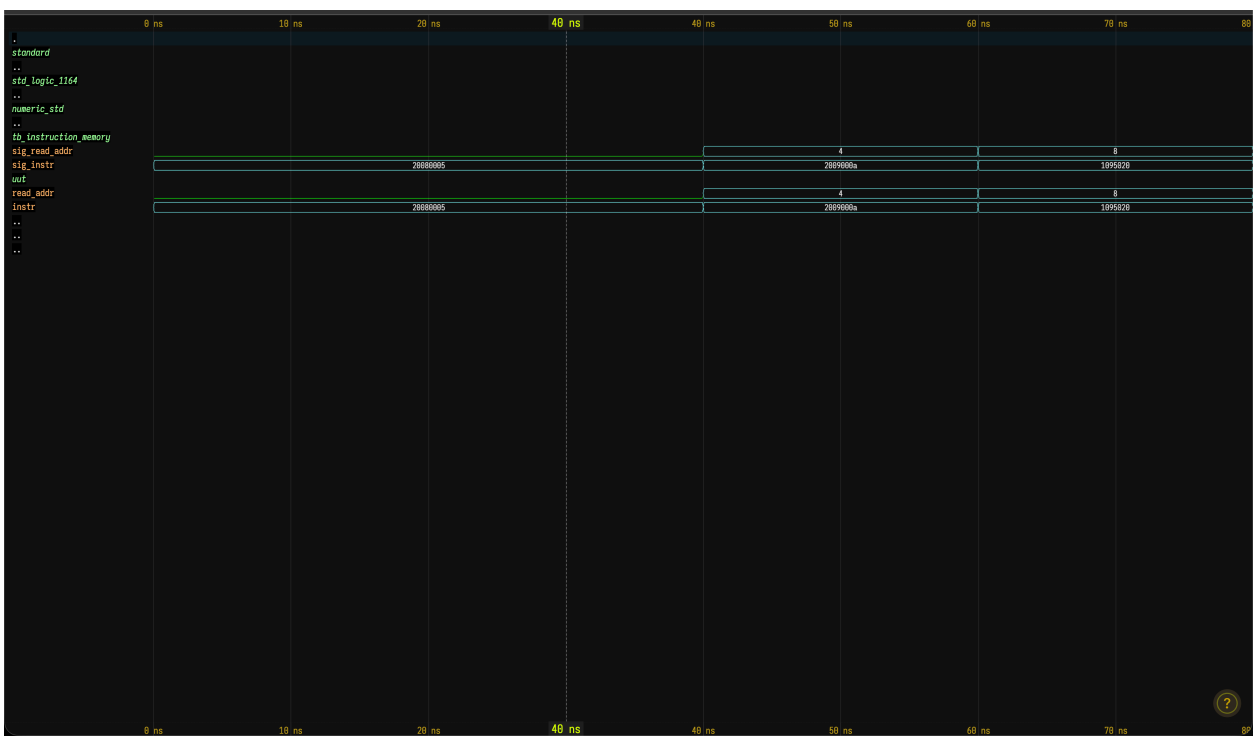
9. **Data Memory (Right): RAM / Data Memory.** Used only by lw and sw to read or write variables to main memory.

10. **Blank Mux (Far Right): Write-Back Multiplexer (MemtoReg).** Chooses whether to send the math result from the ALU or the loaded data from Data Memory back to the Register File to be saved.

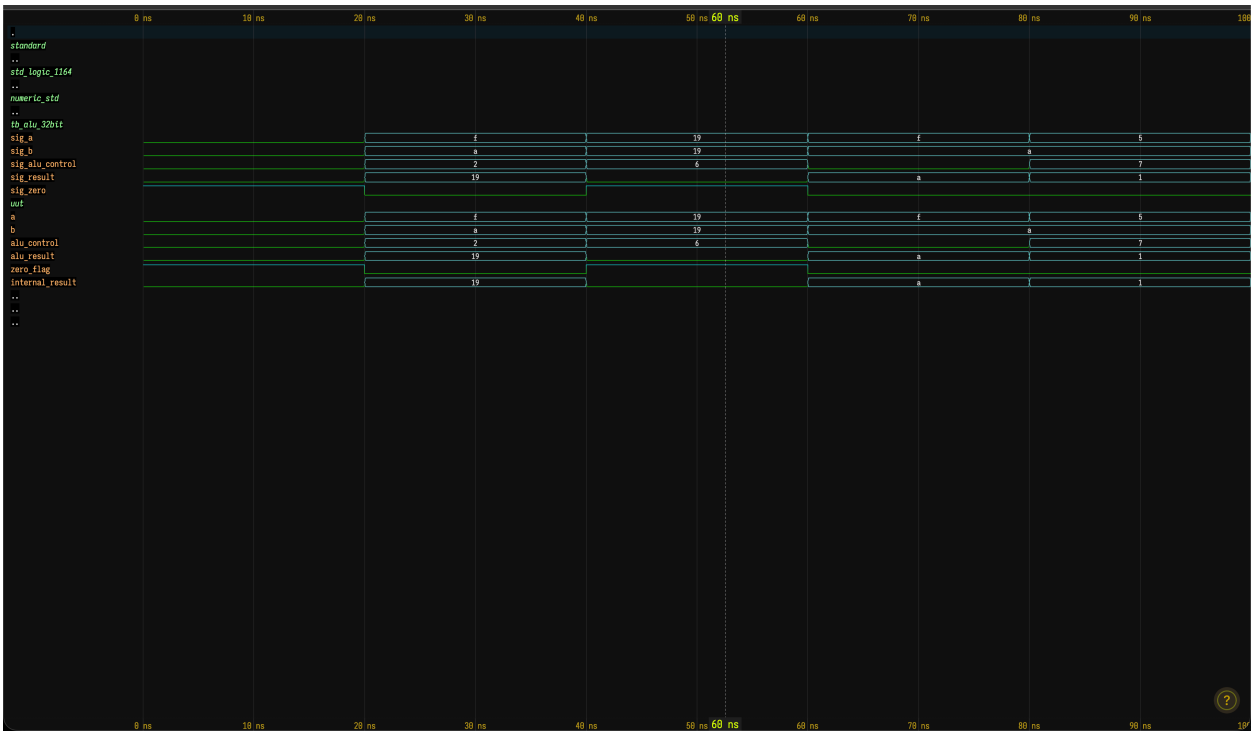
Git branch reconciliation error alongside, terminal where the issue is successfully resolved using the --allow-unrelated-histories flag and pushed to the remote repository.



VS Code workspace showing a VHDL testbench script for a 32-bit ALU and a terminal running GHDL commands to compile and simulate the hardware logic.

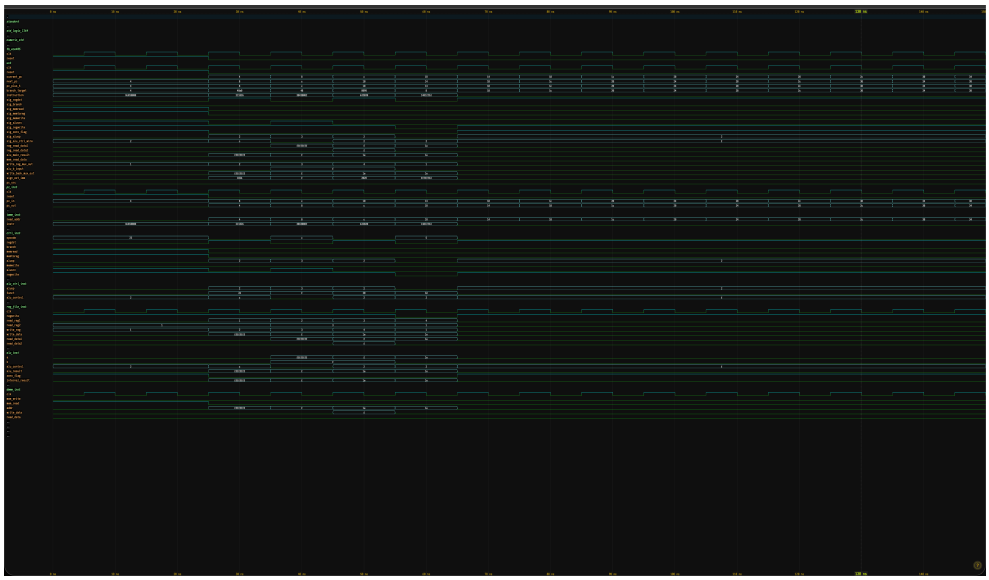


Digital waveform simulation of an instruction memory testbench, displaying 32-bit instruction codes being fetched as the read address increments.

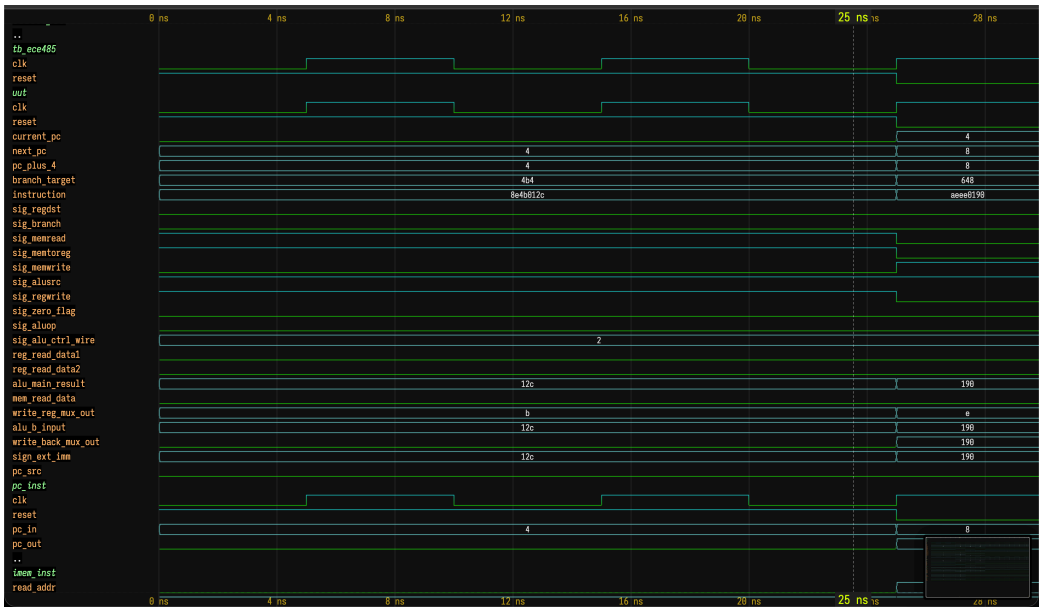


A digital waveform simulation of a 32-bit Arithmetic Logic Unit (ALU) testbench, displaying input operands, control signals, and the resulting mathematical outputs over time.

A digital waveform simulation displaying an expanded view of the MIPS processor's internal components, tracking the control unit, ALU control, and register file signals as the CPU executes a sequence of instructions.



comprehensive digital waveform simulation of a top-level MIPS processor testbench, illustrating the program counter, control signals, and data paths during the execution of an instruction loop.



digital waveform simulation of the top-level MIPS processor testbench, highlighting the initial clock cycles and the exact state transitions of the program counter, instruction fetch, and control signals around the 25-nanosecond mark

10. Sources

1. I**mplementation of the RISC Processor** (PO.Computer (1).pdf). Excerpt detailing the hardware implementation of RISC architectures, including ALU and control unit datapath logic.

2. **Design and Application of RISC Processor** (Design_and_application_of_RISC_processor.pdf). Zaid, M., & Mustajab, P. (2017). *IMPACT 2017*. Paper analyzing multi-cycle 32-bit MIPS processor design, critical path delay reduction, and structural hardware layout.

3. **Computer System Organization and Programming: CPU Design** (06-cpu-notes-bw.pdf). Cornell University, CS 3410 (Spring 2019). Lecture slides detailing MIPS single-cycle datapath construction, instruction fetching, and control logic design. Available at: https://www.cs.cornell.edu/courses/cs3410/2019sp/schedule/slides/06-cpu-notes-bw.pdf

4. **ALL CODE** : https://github.com/eternalshash/mind-overnight/tree/main/code